

Patient Reallocation and Waitlist Reduction in the Norwegian GP System

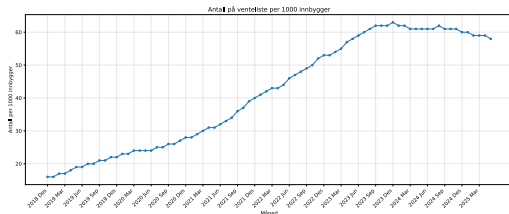
26.06.2026

Lars Møen Haukland

1 Intro

Motivasjon

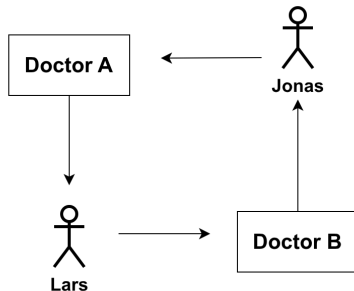
- Fastlegeordningen (2001)
- Vil bytte fastlege til en fastlege med full kapasitet -> venteliste.
- Når det åpner seg ledig kapasitet blir den tildelt pasient som har ventet lengst.
- 77% av alle fastleger med full kapasitet, 2025.
 - ▶ Bergen 97%
 - ▶ Gjennomsnittlig ventetid på 274 dager i 2024, median 137 dager.



Figur 1: Antall på venteliste i Norge, Kilde: helsedirektoratet (2025). *Bytte av fastlege, ventelister og ledige plasser.* helsedirektoratet.no (lest 23.06.2026).

Løsningen?

- Finne bytter mellom pasienter på venteliste.
- 2 pasient bytter eller større ubegrenset lengde sykler.



Figur 2: Eksempel med Lars og Jonas på venteliste til hverandres doktor

Ideen

- Bruke algoritmer for å finne sykler i ventelistene.
- Redusere antall pasienter på ventelistene.
- Redusere ventetid for pasienter på ventelistene.
- Testet algoritmene mot hverandre i simuleringer.
- Analysert hvordan algoritmer med forskjellig mål påvirker resultatene.

2 Bakgrunn

Lignende problemer

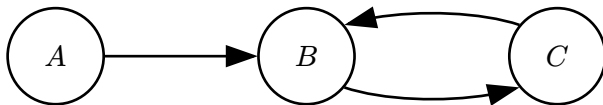
- *Assignment* problemer
- Agenter og objekter - pasienter og leger
- One-to-one
- Many-to-one
- Housing Market
- Kidney exchange
- College Admissions problem

Top Trading Cycles (TTC)

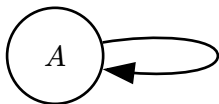
- Algoritme utviklet for å reallokere objekter slik at alle bytter agenter kan gjøre for å få sitt foretrukne objekt blir utført.
- Hver agent har fullstendig prioriteringsliste over alle objekter, sortert etter mest foretrukne.
- Lag en graph $G = (V, E)$, $V =$ agenter
- Lag en kant fra hver agent u til den agenten som eier u sitt mest foretrukne objekt, noteres som $\text{Top}(u)$.
- Finn og utfør en sykel, fjern nodene i syklen fra grafen
- Gjenta med å oppdatere kantene, finn sykel, helt til det ikke er noen noder

Top Trading Cycles (TTC)

- $A = [h_B, h_C, h_A]$
- $B = [h_C, h_A, h_B]$
- $C = [h_B, h_C, h_A]$



Figur 3: Eksempel *TTC* graf.



Figur 4: Eksempel *TTC* graf hvor $\text{Top}(A) = A$.

Egenskaper til TTC

- Strategitrygt
- Individuell rasjonalitet
- Pareto-effektiv

TTC for ventelistene

- Utviklet av Huitfeldt et al.
- Hver pasient har en prioritet.
 - f.eks ventetid eller en funksjon av ventetid.
- $G = (V, E)$, $V = \text{pasienter} \cup \text{leger}$.
- Siden flere pasienter kan ha samme lege, er ikke $\text{Top}(u)$ bare en pasient.
 - Hver pasient har en kant til sin foretrukne lege.
 - Hver lege har en kant til pasienten sin med høyest prioritet.

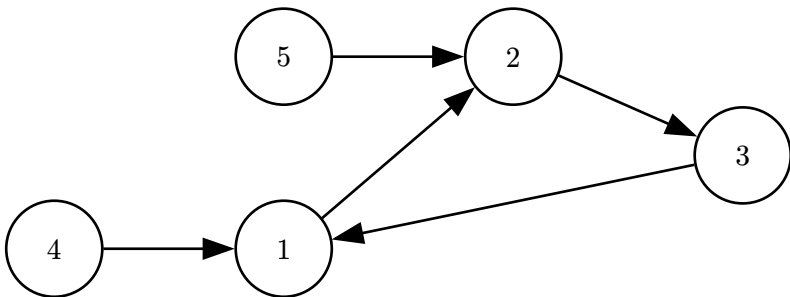
3 GP allocation problem

Definisjon

- Input
 - P : Sett av pasienter
 - D : Sett av leger
 - R : Prioritetsfunksjon $R : P \rightarrow \mathbb{N}$
 - $D_{\text{cur}[i]}, D_{\text{pref}[i]}$: lister som sier nåværende lege og foretrukne lege for pasient i
- Løsning
 - $S \subseteq P$: Sett av pasienter, slik at G_S kan dekkes av ikke-overlappende sykler.

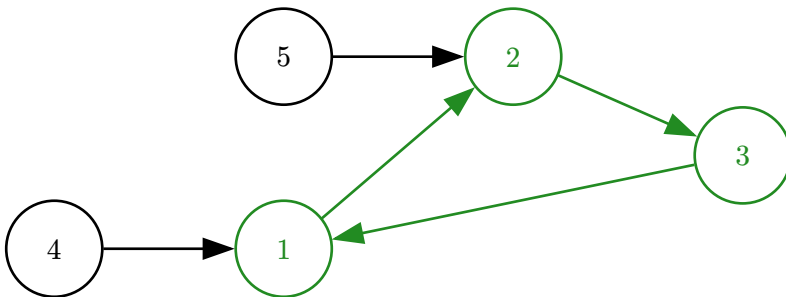
$$G_S = (S, E = \{(a, b) \mid a, b \in S, \text{foretrukne legen til } a \text{ er nåværende lege til } b\})$$

Definisjon (ii)



Figur 5: En instans av *The GP assignment problem*, noder er pasienter og kant (a,b) betyr at a vil ha b sin lege.

Definisjon (iii)



Figur 6: Samme graf som forrige, men løsning i grønt.

Optimal løsning

- Definerer et generell optimeringskriterium $\succ$
- en rangering av løsninger
- optimal løsning dersom ingen annen løsning er mer maksimal under $\succ$
- Lager varianter av dette

Leksikografisk optimalitet

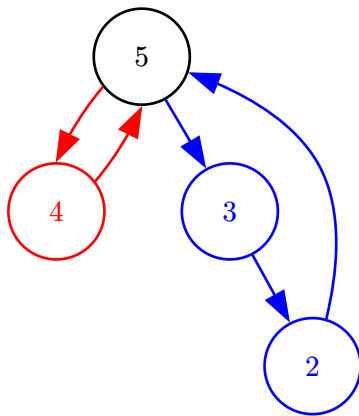
- $\succ_{\text{lex}}$
- Sorter pasienter basert på prioritet slik at: $R(p_1) \geq R(p_2) \geq \dots \geq R(p_n)$, ($n = |P|$)
- $\chi(S) = (b_1, b_2, \dots, b_n) \in \{0, 1\}^n$, $b_i = \begin{cases} 1 & \text{if } p_i \in S \\ 0 & \text{otherwise} \end{cases}$

Definisjon 3.3.1 (Leksikografisk rangering fra prioritet):

$$S \succ_{\text{lex}} S' \Leftrightarrow \chi(S) \text{ er leksikografisk mer maksimal enn } \chi(S')$$

Så, på første index i hvor S og S' er forskjellig, $\chi(S)_i = 1$ og $\chi(S')_i = 0$.

Leksikografisk optimalitet (ii)



Figur 7: En instans med to mulige løsninger

rød: $S = \{5, 4\}$ $\chi(S) = \{1, 1, 0, 0\}$

blå: $S' = \{5, 3, 2\}$ $\chi(S') = \{1, 0, 1, 1\}$

$S \succ_{\text{lex}} S'$

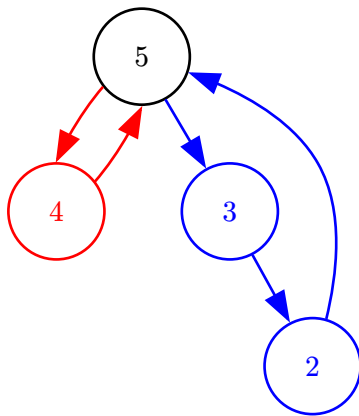
Kardinalitet

- $\succ_{\text{size}}$
- Størrelsen på løsningen

Definisjon 3.4.1 (Kardinalitet rangering):

$$S \succ_{\text{size}} S' \Leftrightarrow |S| > |S'|$$

Kardinalitet (ii)



Figur 8: En instans med to mulige løsninger

rød: $S = \{5, 4\}$ $|S| = 2$

blå: $S' = \{5, 3, 2\}$ $|S'| = 3$

$S' \succ_{\text{size}} S$

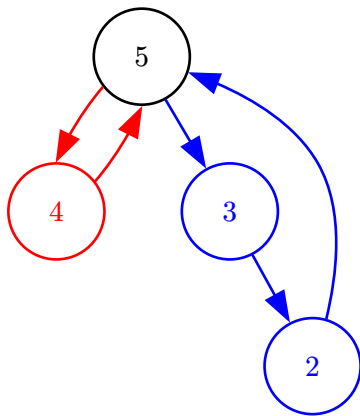
Nytte

- $\succ_{\text{util}}$
- $U(S) = \sum_{p \in S} R(p)$
- Prioritet teller, men kan droppe en høy prioritet pasient for flere lavere.

Definisjon 3.5.1 (Nytte rangering):

$$S \succ_{\text{util}} S' \Leftrightarrow U(S) > U(S')$$

Nytte (ii)



Figur 9: En instans med to mulige løsningen, $R(i) = i$

rød: $S = \{5, 4\}$ $U(S) = 9$

blå: $S' = \{5, 3, 2\}$ $U(S') = 10$

$S' \succ_{\text{util}} S$

Prioritetsfunksjon

- Bestemmer direkte hvilke pasienter som blir inkludert i en maksimal løsning i $\succ_{\text{util}}$ og $\succ_{\text{lex}}$
- Lett å basere på antall dager
- $\text{wait}(a)$ = antall dager a har vært på venteliste
- Lineær: $R(a) = \text{wait}(a)$
- Eksponentiell: $R(a) = k^{\text{wait}(a)}$
 - ▶ k bestemmer hvor mye prioritet er vektet
 - ▶ $k = 1$ alle prioriteter er 1 $\rightarrow \succ_{\text{util}}$ blir $\succ_{\text{size}}$
 - ▶ $k = 2$ hvis $\text{wait}(a)$ er unik for alle pasienter $\rightarrow \succ_{\text{util}}$ blir $\succ_{\text{lex}}$

4 Algoritmer

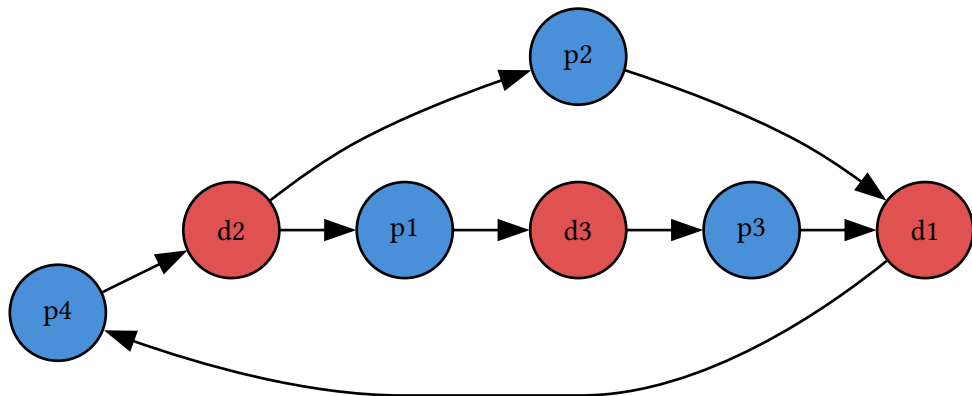
Top Trading Cycles (TTC)

- Tatt fra replikasjonspakken
- Python oversatt til rust
 - Kjøretid ikke påvirket av språk

Greedy DFS

- $G = (V, E), \quad V = P \cup D$
- $E = \left\{ (p_i, D_{\text{pref}[i]}) \right\} \cup \left\{ (D_{\text{cur}[i]}, p_i) \right\}$
 - ▶ Pasienter peker på sin foretrukne lege
 - ▶ Leger peker på alle sine pasienter
- For hver pasient i synkende prioritet
 - ▶ Prøv å finn en sykel med denne pasienten, DFS
 - På hver lege gå til pasient med høyeste prioritet først
 - ▶ Gi alle pasientene i syklen sin foretrukne lege og fjern fra grafen
- Hvis ingen sykel finnes for en pasient, kan den markeres som «stuck»
- $O(|P| (|D| + |P|))$

Greedy DFS (ii)



Figur 10: Eksempel graf G hvor *Greedy DFS* ville valgt en suboptimal løsning.
 px betyr pasient x har prioritet x .

Minimum Cost Circulation

- Input: $G = (V, E)$ hvor G er en rettet multigraf
- Kantene er tripler: (v, w, i)
 - Har kostnad $c(v, w, i) \in \mathbb{Z}$
 - Og kapasitet $u(v, w, i) \geq 0$
- Finne en circulation f som setter verdi på hver kant som tilfredstiller:

$$0 \leq f(v, w, i) \leq u(v, w, i) \quad \forall (v, w, i) \in E$$

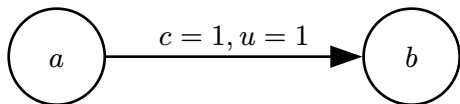
$$\sum_{(q,w,i) \in \text{in}(w)} f(q, w, i) = \sum_{(w,v,j) \in \text{out}(w)} f(w, v, j) \quad \forall w \in V$$

$$\text{cost}(f) = \sum_{(v,w,i) \in E} c(v, w, i) f(v, w, i)$$

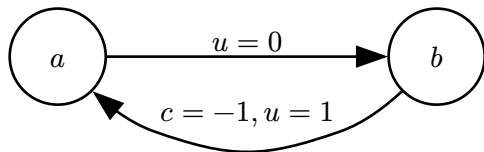
- Finne circulation f^* med minst kostnad

Residual graf

- Omskrivning av grafen basert på f
- Nodene endres ikke fra original grafen
- For hver kant (v, w, i) lag to kanter
 - ▶ (v, w, i) fremover kant
 - samme kostnad som i original
 - $u_{f(v,w,i)} = u(v, w, i) - f(v, w, i)$
 - ▶ (w, v, i) baklengs kant
 - $-c(v, w, i)$
 - $u_{f(w,v,i)} = f(v, w, i)$



Figur 11: Kant $a \rightarrow b$ har kostnad $c = 1$ og kapasitet $u = 1$.



Figur 12: Residual grafen $G(f)$ etter at $f(a, b) = 1$, fremover kanten er mettet og bakover kanten har kapasitet 1.

Optimal circulation

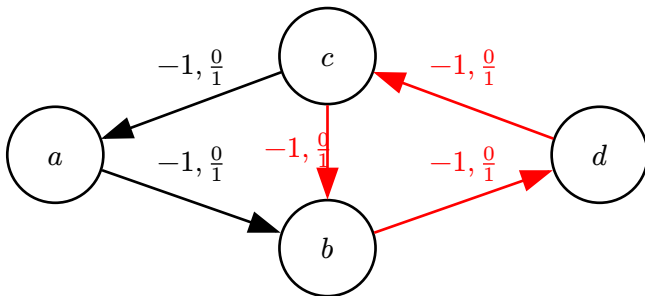
Theorem 4.5.1 (Negative Cycle Optimality Theorem): En circulation f^* er en optimal løsning til *Minimum Cost Circulation* problemet hvis og bare hvis residual grafen $G(f^*)$ ikke har noen negativ-kostnad rettede sykler.

- I en instanse med bare positive kostnader, $f^*(e) = 0 \quad \forall e \in E$
- Vi setter kostnadene til å være negative, da finner vi også løsningen med maksimal total circulation

$$\sum_{(v,w,i) \in E} f(v, w, i)$$

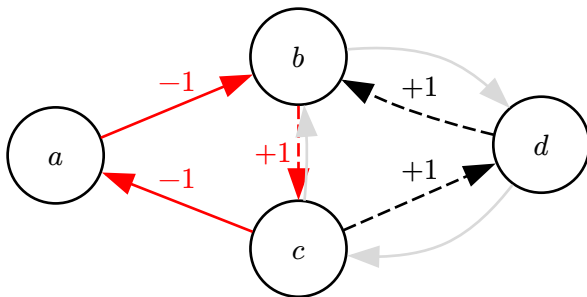
Cycle Cancelling

- Starter med 0 circulation: $f^0(e) = 0 \quad \forall e \in E$
- Så lenge det finnes en negativ sykel i residual grafen
 - ▶ Kanseller den: Øk flyt på hver kant i syklens med den minste kapasiteten til en av kantene i syklens



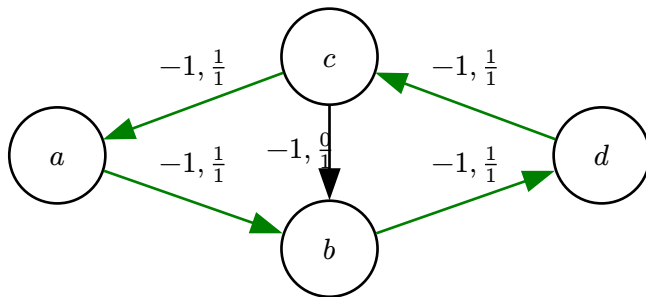
Figur 13: Start: $f = 0$. Ved $f = 0$ er residualgrafen lik originalen. Negativ sykel $d \rightarrow c \rightarrow b \rightarrow d$ har kostnad -3 . Merking: $c, f/u$.

Cycle Cancelling (ii)



Figur 14: Residualgrafen $G(f)$: mettede fremoverkanter vises som grå, baklengskanter (stiplet) har kostnad $+1$. Negativ sykel $a \rightarrow b \rightarrow c \rightarrow a$ har kostnad -1 .

Cycle Cancellation (iii)



Figur 15: Etter andre kansellering: kostnad -4 , sirkulasjonen $a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$ – optimal.

Cycle cancelling kjøretid

- $O(nm^2CU)$, $n = |V|$, $m = |E|$
 - C : maks kostnad på en kant
 - U : maks kapasitet på en kant
- Pseudo-polynom
 - For en av våre algoritmer setter vi kostnad basert på R
- *Mean Cycle Cancellation*

Cycle Cancellation for kardinalitet

- $G = (V, E)$, $V = D$
- Kant (v, w, i) dersom det finnes minst en pasient som vil bytte fra lege v til lege w
- Kostnad $c(v, w, i)$ er -1
- Kapasitet er antall pasienter som vil ha det byttet
 - $u(v, w, i) = 5$: fem pasienter vil bytte fra v til w
- Grafen lagrer ingenting om prioritet
- Etter *Cycle Cancellation* har kjørt på grafen får vi bare vite hvor mange som kan byttes på hver kant
 - Hvis k pasienter kan byttes, velge de k som har ventet lengst
- Negativ kostnad gjør at vi finner optimal løsning under $\succ_{\text{size}}$
- Kjøretid: $O(nm^2CU) \rightarrow O(nm^2 |P|)$

Cycle Cancelling for Nytte

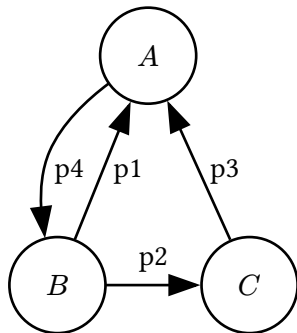
- $G = (V, E)$, $V = D$
 - Kant (v, w, i) dersom det finnes én pasient i som vil fra lege v til w
 - En kant per pasient
 - Kostnad $c(v, w, i) = -R(p_i)$
 - Kapasitet $u(v, w, i) = 1$ én pasient per kant
-
- Etter *Cycle Cancelling* har kjørt på grafen har vi flyt på noen kanter som betyr at pasientene som tilhører kan få sin foretrukne lege.
 - Optimal løsning under $\succ$
 - Kjøretid: $O(nm^2CU) \xrightarrow{\text{util}} O(nm^2 \max R(p))$
 - pseudo-polynom
 - Max prioritet i våre simuleringer
 - Lagret som 128 bit heltall. $R(a) = 2^{\frac{\text{wait}(a)}{10}}$
 - Maks 1010 dager

Cycle Cancelling for $\succ_{\text{lex}}$

- Bruker fortsatt cycle cancelling teknikken men med litt endring
- $G = (V, E), V = D$
- Kant (v, w, i) dersom det finnes én pasient i som vil fra lege v til w .
- Bryr oss ikke om kostnad
- Kapasitet $u(v, w, i) = 1$

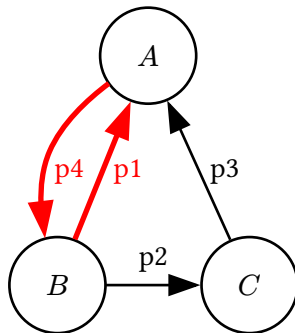
- For hver pasient p_i i synkende prioritet
 - La $e = (v, w, i)$ være kanten til p_i
 - Hvis $f(e) = 1$: Legg p til i løsningen og slett e og residual kanten dens fra grafen
 - Ellers:
 - Hvis det finnes en sti Q fra w til v i $G(f)$, bruker BFS
 - Det finnes sykel som inneholder p_i da må den være med i løsningen
 - Øk flyt med en på hver kant i Q , legg p_i til løsningen og slett e og residual kanten dens

Cycle Cancelling for $\succ_{\text{lex}}$ (ii)



Figur 16: Noder p_x har prioritet x .

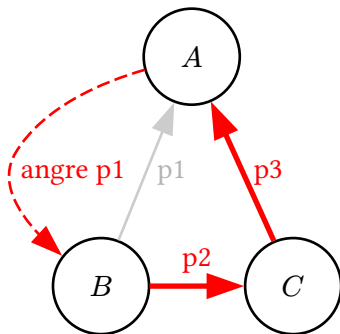
Cycle Cancelling for $\succ_{\text{lex}}$ (iii)



Figur 17: Steg 1: behandle p_4 . Vi finne sykel $A \rightarrow B \rightarrow A$ via p_1 . p_4 legges til i løsningen, kanten slettes.

$$S = \{p_4\}$$

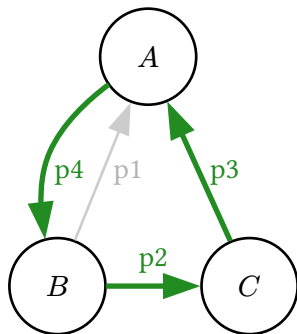
Cycle Cancellation for $\succ_{\text{lex}}$ (iv)



Figur 18: Steg 2: behandle p_3 . p_4 er allerede bundet (kant fjernet). Eneste vei $A \rightarrow C$ bruker baklengskanten $A \rightarrow B$ som angreir p_1 . Sykelen $C \rightarrow A \rightarrow B \rightarrow C$ legger p_3 i løsningen og øker flyt på p_2 .

$$S = \{p_4, p_3\}$$

Cycle Cancellation for $\succ_{\text{lex}}(\mathbf{v})$



Figur 19: Resultat: den store sykkelen løser p_4, p_3, p_2 . leksikografisk optimal.

- Kjøretid: $O((n + m) |P|)$
 - Går over alle pasienter: $|P|$ iterasjoner
 - For hver pasient så søker vi med BFS $(n + m)$ iterasjoner

5 Eksperimenter og resultater

Simulering og data

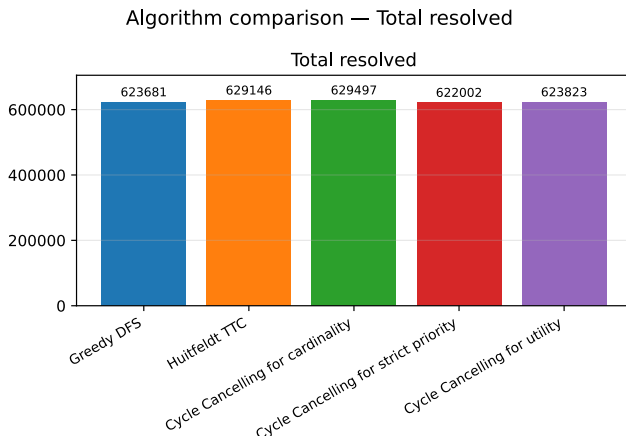
- Ikke mulighet for ekte data
 - Tilfeldig generert
- Simulerer en mindre instans men proporsjonell til Norges tall.
- To simuleringer
 - 100 år: se utviklingen av ventelistestørrelse
 - 2 år: Inkludere eksponentielle prioriteringsfunksjoner
- For hver dag:
 - Øke ventetiden til pasienter med 1
 - Kjøre algoritmen og fjerne pasienter som løses
 - Legge til nye pasienter på ventelister

Metrikker

- Antall hjulpet
- Størrelse på ventelisten
- Vente tid
 - Maks
 - Gjennomsnitt
- Kjøretid

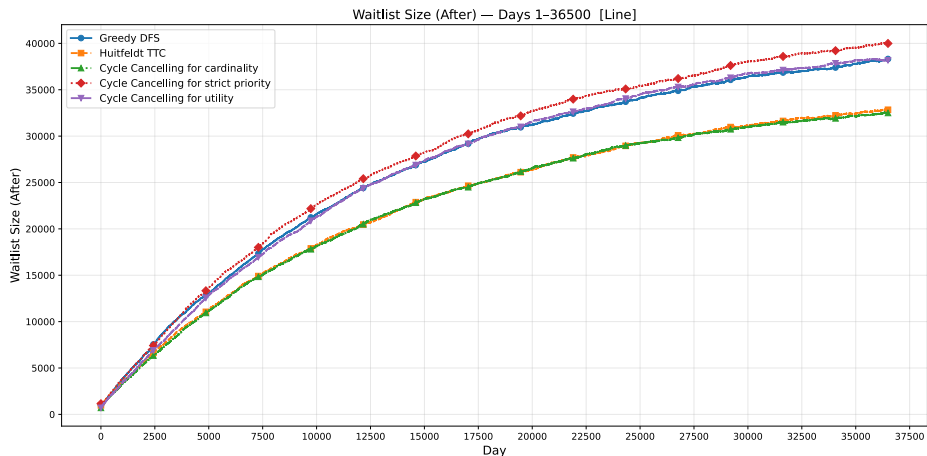
Antall hjulpet

- Lite variasjon



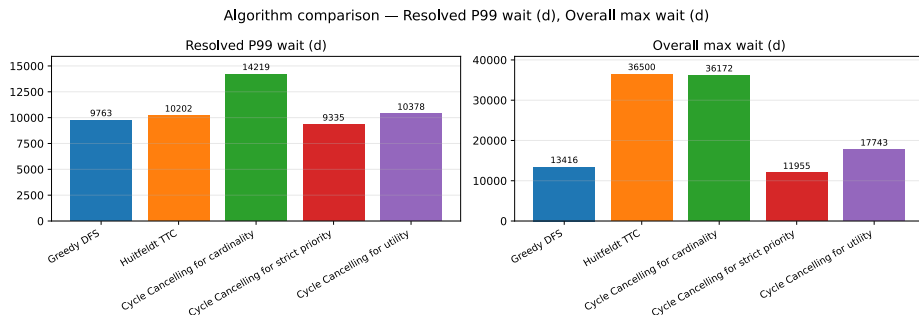
Figur 20: Totalt antall legebytter utført

Størrelse på ventelisten



Figur 21: Totalt antall pasienter på venteliste over tid

Maks vente tid



Figur 22: Maks ventetid for løste bytter, uten de 1% lengste på venstre siden og totalt max ventetid inkludert pasienter som ikke fikk byttet på høyre.

Gjennomsnittlig vente tid

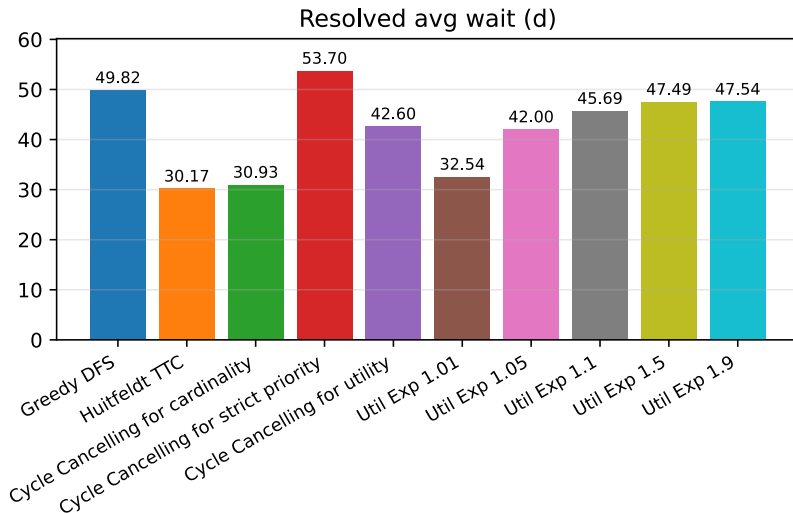
Algorithm comparison — Resolved avg wait (d)



Figur 23: Gjennomsnittlig ventetid for bytter som ble løst

Gjennomsnittlig vente tid (ii)

Algorithm comparison — Resolved avg wait (d)



Konklusjon

- Ikke representabel simulering ift Norge.
 - Ingen pasienter som dør eller blir født.
 - Økende venteliste ville ikke skjedd i et reelt system.
 - Først kjøre ventelistesteg hvor alle ledige plasser blir gitt.
- Viser fortsatt hvordan algoritmenes prioriteringer endrer resultat.

Videre arbeid:

- Teste med ekte data
- Dynamisk algoritme, maksimerer $\succ_{\text{size}}$ så lenge det ikke er noen pasienter som har ventet i mer enn x dager, ellers maksimer $\succ_{\text{lex}}$.
- Analysere om noen kan utnytte algoritmene for å få en urettferdig fordel.
- Bruke *Mean Cycle Cancelling* for polynomisk kjøretid for *Cycle Cancelling for Nytte*

6 Takk for meg, spørsmål?